KURA LIVE

System Architecture, Operations, and User Reference Guide

Document Field	Value
System Name	KURA LIVE - Kenya Unified Results Architecture
Primary Purpose	Election monitoring, verification, live stream intelligence, and discrepancy management
Deployment Context	Next.js 14 local deployment with Supabase scaffolding and local fallback persistence
Prepared For	Engineers, election operators, field agents, administrators, and support teams
Workspace	C:\Users\VincentOmbok\Documents\Kura27project
Current Runtime	Localhost deployment on port 3000

How To Use This Guide

This guide is written for two audiences at once. Engineers can use it to understand architecture, integration points, configuration, and maintenance. Non-technical users can use it to learn how the system works operationally and how to complete common tasks.

- If you need a quick orientation, start with System Overview, Features and Functionality, and User Guide.
- If you are deploying, troubleshooting, or extending the platform, read Architecture and Design, Technical Specifications, Installation and Setup, and Maintenance and Support.
- If you are monitoring live election operations, focus on the Agent Workflow, Command Centre, Comparison Engine, Alerts, and Troubleshooting sections.

Table Of Contents

- 1. System Overview
- 2. Architecture and Design
- 3. Technical Specifications
- 4. Features and Functionality
- 5. Installation and Setup
- 6. Configuration Options
- 7. User Guide
- 8. API Reference and Integration Points
- 9. Troubleshooting and FAQs
- 10. Maintenance and Support

1. System Overview

KURA LIVE is an election monitoring and verification platform designed to observe live polling-station result announcements, convert spoken announcements into structured tallies, compare those tallies against scanned Form 34A results and official IEBC figures, and raise discrepancies in real time.

The platform does not host or originate livestreams. Instead, field agents use their own TikTok accounts, YouTube accounts, or approved external tools to go live from polling stations. KURA LIVE then monitors those streams, captures shared tab audio, transcribes the result announcement workflow, and feeds the extracted figures into a three-source verification engine.

Core Purpose

- Give field agents a structured way to register and monitor live polling-station broadcasts.
- Transcribe spoken vote announcements with OpenAI Whisper and parse them into candidate vote counts.
- Compare live audio tallies with OCR-extracted Form 34A tallies and official IEBC values.
- Flag any discrepancy of one vote or more and surface it nationally in a command centre.
- Support both live election operations and fallback demonstration/testing workflows on localhost.

Key Capabilities

Capability	What It Means In Practice
Field stream registration	Agents paste TikTok, YouTube, or RTMP-related stream URLs into KURA LIVE and bind them to stations.
Live audio monitoring	Operators use browser tab audio capture to ingest spoken announcements without illegally ripping source streams.
Realtime transcription	Captured audio is chunked and sent to Whisper progressively, not only after a full clip ends.
Three-source verification	Audio AI, Form 34A OCR, and IEBC official figures are stored and compared per candidate per station.
Alert escalation	The system automatically creates open alerts when deltas are detected and supports escalation or dismissal workflows.
National command centre	A geospatial and operational dashboard shows live streams, verification status, alerts, and tally movement nationally.

2. Architecture and Design

KURA LIVE follows a single-repository Next.js application model. The frontend, API routes, UI state, and local fallback persistence live in the same codebase. Supabase is scaffolded as the long-term system of record, while a local JSON persistence layer is used to keep the demo and localhost environment stable even when hosted schema support is incomplete.

High-Level Component Model

```
Field Agent Live Stream URL
    -> Agent Portal (Stream Management + Transcription)
    -> Browser Tab Audio Capture
```

```
-> Chunked Live Transcription API
-> Whisper Processing + Vote Parser
-> Tally Persistence
-> Reconciliation Engine
-> Alerts + Verification Events
-> Command Centre + Map + Stream Wall + Comparison Views
```

Major Runtime Layers

Layer	Main Responsibilities	Representative Files
Presentation layer	Pages, route layouts, operator views, agent views, map view, stream wall, and public results UX.	src/app/*, src/components/*
Client state layer	Realtime snapshot hydration, tallies, alerts, and toast notifications.	src/store/websocketStore.ts, src/store/tallyStore.ts, src/store/alertStore.ts, src/store/toastStore.ts
API layer	Stream registration, transcription, forms processing, IEBC sync, alerts, tally comparison, and command-centre snapshots.	src/app/api/*
Domain logic layer	Vote parsing, candidate matching, discrepancy scoring, live session control, command-centre snapshot building.	src/lib/*.ts
Persistence layer	Supabase scaffolding for relational tables plus local JSON fallback state.	src/lib/supabase.ts, src/lib/verificationDemo.ts, supabase/migrations/*
Security and routing layer	Session cookie parsing, path authorization, middleware redirects, and role separation.	src/lib/session.ts, src/lib/access.ts, src/middleware.ts

Primary User Experiences

- Agent Portal: Field-facing workspace for stream monitoring, live transcription, Form 34A processing, comparison, and alerts.
- Admin Portal: Administrative and oversight workspace for broader campaign or operational controls.
- Command Centre: National monitoring dashboard showing stations, streams, alerts, and aggregation behavior.
- Public Results Portal: Public-facing view for results and map context.

3. Technical Specifications

Category	Specification
Frontend framework	Next.js 14 with App Router
Language	TypeScript
Styling	Tailwind CSS
UI state	Zustand
Forms and validation	React Hook Form and Zod
Animation	Framer Motion

Charting	Recharts
Video support	react-player plus custom stream preview handling
Map stack	Leaflet and react-leaflet with SSR-safe dynamic imports
Database scaffolding	Supabase PostgreSQL
AI transcription	OpenAI Whisper API
Deployment mode validated here	Localhost deployment via Next.js production server

Important Code Modules

Module	Role In The System
src/components/agent/TranscriptionPanel.tsx	Handles demo audio, live tab capture, chunk uploads, transcript display, and extracted vote updates.
src/components/agent/StreamPanel.tsx	Manages stream URL inputs, platform preference, preview, and live stream context for agents.
src/components/command/LiveMapPanel.tsx	National map view with station state, verification markers, and stream/alert overlays.
src/components/command/StreamWall.tsx	Displays active live sessions with operational metrics and quick feed access.
src/lib/verificationDemo.ts	Local fallback persistence, seeded demo state, alert synchronization, and comparison-row generation.
src/lib/commandCenter.ts	Builds the normalized command-centre snapshot consumed by the national dashboard.
src/app/api/transcription/live/route.ts	Processes 5 to 10 second live audio chunks and returns progressive transcript/tally updates.
src/app/api/streams/route.ts	Registers streams, stores or mirrors live sessions, and updates lifecycle state.

4. Features and Functionality

4.1 Stream Management

Agents can supply TikTok Live URLs, YouTube Live URLs, or platform URLs associated with their polling station. The system validates and stores the stream association, then opens a monitoring preview without pretending to own the underlying stream provider.

- Primary operator action: paste a live URL and choose the platform context.
- System action: register the stream session, bind it to agent identity and station code, and expose it to the command centre.
- Operational limitation: the system uses browser-shared audio capture instead of extracting media directly from restricted platforms.

4.2 Live Audio Capture and Transcription

The agent starts live monitoring from the Transcription panel. The browser asks for tab or system audio access. Once approved, KURA LIVE uses the MediaRecorder API to create progressive audio chunks, normally on an 8-second cadence in the current implementation. Each chunk is uploaded to the live transcription route and processed independently.

Step	Current Behavior

Capture permission	User shares a browser tab with audio enabled.
Chunk creation	MediaRecorder emits a new audio blob approximately every 8 seconds.
API handling	The chunk is sent to /api/transcription/live with session, station, and timing metadata.
AI processing	OpenAI Whisper is used when configured; otherwise or on failure, the system falls back gracefully to deterministic parsing behavior.
UI update	Transcript text, extracted votes, tallies, and alerts update progressively in the agent portal.

4.3 Vote Extraction and Parsing

The parsing layer converts freeform announcements into structured results. It supports English and partial Swahili phrases, resolves candidate aliases, suppresses duplicates, and assigns confidence scores. Rejected votes are also detected when phrased explicitly.

- Example English pattern: John Kariuki received 120 votes.
- Example Swahili pattern: Mary Wambui amepata kura 115.
- Edge behavior: if Whisper fails, the system can still parse fallback transcript text and keep the operator workflow moving.

4.4 Form 34A OCR Flow

The Form 34A module accepts bundled samples or user uploads and maps them into OCR-style extracted results for the demonstration environment. These results are stored as the Form 34A verification source and immediately enter the comparison matrix.

4.5 IEBC Official Source Integration

The current implementation uses a mock official dataset to simulate the IEBC portal feed. The API persists those values as the IEBC source so that three-way comparison can happen deterministically in the local environment.

4.6 Comparison and Alerts

For every candidate at a station, the system compares Audio AI, Form 34A, and IEBC tallies. Any discrepancy of one vote or more creates an alert. Severity is warning for deltas from one to four and critical for five or more.

- Open alerts appear in the agent portal and command centre.
- Operators can escalate or dismiss alerts from the alerts panel.
- Verification events and station activity are also tracked in local state and command-centre snapshots.

4.7 Command Centre and Map View

The command centre at /dashboard is the national monitoring plane. It combines a live station map, tally summary, incident panel, stream wall, AI monitoring panel, and an operational right rail. The map view at /map-view gives a larger, map-focused monitoring screen.

Dashboard Area	Operational Use
Live map	Shows station status, stream presence, verification health, and

	alert counts geographically.
Stream wall	Lists monitored streams with platform, signal quality, latency, and operator access controls.
Results tally	Aggregates candidate performance and turnout indicators for top-level monitoring.
Incident and alert panels	Surface discrepancies and operational problems needing review.
Right rail	Displays high-level system health, turnout, counties reporting, and API/stream health indicators.

5. Installation and Setup

1. Install dependencies with npm install.
2. Create or update .env.local with the required environment variables.
3. Run npm run build to generate a production build.
4. Run npm run start -- --port 3000 for the local production server, or npm run dev for development.
5. Open http://localhost:3000 in a browser.

Required Local Commands

```
npm install
npm run lint
npm run typecheck
npm test
npm run build
npm run start -- --port 3000
```

Supabase Preparation

Two migration files are present for the verification and live monitoring model. These should be applied to the Supabase project if the hosted database is intended to replace local fallback persistence.

- supabase/migrations/20260504_verification_pipeline.sql
- supabase/migrations/20260511_live_monitoring_pipeline.sql

6. Configuration Options

Variable	Purpose	Required
NEXT_PUBLIC_SUPABASE_URL	Supabase project URL used by browser and server-side client scaffolding.	Yes
NEXT_PUBLIC_SUPABASE_ANON_KEY	Public Supabase anon key for client-side access.	Yes
SUPABASE_SERVICE_ROLE_KEY	Server-side elevated Supabase key for privileged operations.	Yes for full DB integration
NEXT_PUBLIC_APP_URL	Base application URL used for local routing and references.	Yes
OPENAI_API_KEY	Used for Whisper transcription of uploaded and live chunked audio.	Yes for real Whisper

		use

Important security rule: service keys and OpenAI keys must never be exposed in browser code, screenshots, or public documentation. Only server routes should use them.

7. User Guide

7.1 Field Agent Workflow

6. Sign in to the agent portal.
7. Open the Live Stream tab and register or preview the monitoring URL.
8. Open the Transcription tab.
9. Use Open agent TikTok, Open agent YouTube, or paste a live URL.
10. Click Start live capture and share the correct browser tab with audio enabled.
11. Allow at least one chunk cycle so transcript and tally updates appear.
12. Upload or process a Form 34A sample from the Form 34A tab.
13. Fetch IEBC tallies.
14. Review Comparison and Alerts.

7.2 Command Centre Operator Workflow

15. Open /dashboard after authentication.
16. Review the map for reporting stations, discrepancy hotspots, and stream density.
17. Open streams from the stream wall if a station needs deeper review.
18. Use the tally panel for a quick national summary.
19. Review incidents and alerts for escalation priority.
20. Use the full map at /map-view for focused geospatial monitoring.

7.3 Public Viewer Workflow

21. Open /results to view public-facing results and map context.
22. Use search and filtering to inspect station or constituency context.
23. Use /results-tally for a denser tally-specific presentation.

8. API Reference and Integration Points

Endpoint	Method	Purpose
/api/streams	GET / POST / PATCH	List streams, register new monitoring sessions, and update lifecycle state.
/api/transcription	POST	Process uploaded or demo audio as a non-live transcription flow.
/api/transcription/live	POST	Process chunked live audio with session metadata and return progressive transcript updates.
/api/forms	GET / POST	List form uploads and process Form 34A samples or uploads.
/api/iebc/[station]	GET	Fetch and optionally persist official IEBC comparison data.
/api/tallies	GET / POST	Read tally entries or persist source-specific

		values.
/api/alerts	GET / PATCH	Read alerts and update alert status.
/api/verification/compare	GET / POST	Read comparison rows and station verification stats.
/api/command-centre	GET	Return the normalized national dashboard snapshot.

Typical Live Transcription Payload

```
station_code=KE-047-290-0001
agent_id=agent-001
session_id=<uuid>
chunk_index=0
started_at=<unix ms>
ended_at=<unix ms>
sample_id=audio-westlands-official
audio=<binary webm blob>
```

9. Troubleshooting and FAQs

Common Issues

Problem	Likely Cause	Recommended Action
Command centre returns an error	Persisted local demo state uses an older schema.	Restart the app after the state loader self-heals, or reset the local demo bootstrap endpoint.
No live transcript appears	Tab audio was not shared, browser denied capture, or the source tab has no audible content.	Restart live capture and ensure the selected tab is shared with audio enabled.
Whisper transcription does not use the real API	OPENAI_API_KEY is missing or the upstream request failed.	Verify the environment variable and check network/API availability.
Form processing works but does not reflect a real OCR engine	The localhost environment uses sample-based extraction fallback.	Apply Supabase and OCR backend integrations if production OCR is required.
Dashboard is accessible in raw HTTP tests but not in the browser session	The browser session may not be authenticated or cookies may have expired.	Log in again and confirm the correct role cookie is present.
Build or typecheck runs out of memory	Node heap was too low for this workspace.	Use the package scripts provided; they already raise the heap limit to 4096 MB.

Frequently Asked Questions

- Does KURA LIVE host the stream? No. KURA LIVE monitors streams owned and started elsewhere.
- Can the system directly rip TikTok media? No. The supported compliance model is browser-shared audio capture.
- Can agents see the command centre? Yes, the shared command centre is accessible to authenticated agents for monitoring.

- Is Supabase fully authoritative today? Not in this localhost rescue state. The code supports Supabase, but local JSON fallback persistence remains active when hosted schema support is incomplete.
- Can this guide be exported to PDF? Yes. The generated Word document is designed for Word or PDF export.

10. Maintenance and Support

Routine Maintenance

- Re-run lint, typecheck, tests, and build after any code change.
- Apply pending Supabase migrations if the hosted database is being used as the authoritative store.
- Rotate secrets such as OpenAI keys and service-role keys according to your security policy.
- Review the command-centre snapshot output when adding new live-session or verification fields.
- Keep browser-capture flows tested in the target browser versions used by field agents.

Operational Escalation Path

24. Field agent problem: confirm login, station assignment, and tab-audio permissions.
25. Application problem: check API route responses, alert/tally state, and local persistence health.
26. Infrastructure problem: check environment variables, Supabase connectivity, and Whisper API availability.
27. Production support problem: promote issue details to the engineering team with route, station, session, and transcript timing context.

Validation Baseline

At the time of this documentation build, the validated engineering checks are npm run lint, npm run typecheck, npm test, and npm run build. The local production server is expected to run on port 3000.

Appendix A: Key Local Routes

Route	Audience	Purpose
http://localhost:3000/	All users	Landing page and navigation starting point.
http://localhost:3000/login	Authenticated users	Main login route for shared credentials.
http://localhost:3000/agent-login	Field agents	Dedicated agent login flow.
http://localhost:3000/agent	Field agents	Agent portal for live stream monitoring and verification.
http://localhost:3000/dashboard	Agents and admins	Shared command centre.
http://localhost:3000/admin	Admins only	Administrative oversight portal.
http://localhost:3000/map-view	Agents and admins	Full-page national map monitoring view.
http://localhost:3000/results	Public and operators	Public results and contextual monitoring view.